Chapter 11. High-Level Languages

In this chapter we will begin to look at our first "real-world" programming language. Assembly language is the language used at the machine’s level, but most people find coding in assembly language too cumbersome for everyday use. Many computer languages have been invented to make the programming task easier. Knowing a wide variety of languages is useful for many reasons, including

- Different languages are based on different concepts, which will help you to learn different and better programming methods and ideas.
- Different languages are good for different types of projects.
- Different companies have different standard languages, so knowing more languages makes your skills more marketable.
- The more languages you know, the easier it is to pick up new ones.

As a programmer, you will often have to pick up new languages. Professional programmers can usually pick up a new language with about a weeks worth of study and practice. Languages are simply tools, and learning to use a new tool should not be something a programmer flinches at. In fact, if you do computer consulting you will often have to learn new languages on the spot in order to keep yourself employed. It will often be your customer, not you, who decides what language is used. This chapter will introduce you to a few of the languages available to you. I encourage you to explore as many languages as you are interested in. I personally try to learn a new language every few months.

Compiled and Interpreted Languages

Many languages are *compiled* languages. When you write assembly language, each instruction you write is translated into exactly one machine instruction for processing. With compilers, a statement can translate into one or hundreds of

machine instructions. In fact, depending on how advanced your compiler is, it might even restructure parts of your code to make it faster. In assembly language what you write is what you get.

There are also languages that are *interpreted* languages. These languages require that the user run a program called an *interpreter* that in turn runs the given program. These are usually slower than compiled programs, since the interpreter has to read and interpret the code as it goes along. However, in well-made interpreters, this time can be fairly negligible. There is also a class of hybrid languages which partially compile a program before execution into byte-codes. This is done because the interpreter can read the byte-codes much faster than it can read the regular language.

There are many reasons to choose one or the other. Compiled programs are nice, because you don’t have to already have an interpreter installed in the user’s machine. You have to have a compiler for the language, but the users of your program don’t. In an interpreted language, you have to be sure that the user has an interpreter installed for your program, and that the computer knows which interpreter to run your program with. However, interpeted languages tend to be more flexible, while compiled languages are more rigid.

Language choice is usually driven by available tools and support for programming methods rather than by whether a language is compiled or interpretted. In fact many languages have options for either one.

High-level languages, whether compiled or interpreted, are oriented around you, the programmer, instead of around the machine. This opens them up to a wide variety of features, which can include the following:

- Being able to group multiple operations into a single expression
- Being able to use "big values" - values that are much more conceptual than the 4-byte words that computers normally deal with (for example, being able to view text strings as a single value rather than as a string of bytes).

- Having access to better flow control constructs than just jumps.
- Having a compiler to check types of value assignments and other assertions.
- Having memory handled automatically.
- Being able to work in a language that resembles the problem domain rather than the computer hardware.

So why does one choose one language over another? For example, many choose Perl because it has a vast library of functions for handling just about every protocol or type of data on the planet. Python, however, has a cleaner syntax and often lends itself to more straightforward solutions. It's cross-platform GUI tools are also excellent. PHP makes writing web applications simple. Common LISP has more power and features than any other environment for those willing to learn it. Scheme is the model of simplicity and power combined together. C is easy to interface with other languages.

Each language is different, and the more languages you know the better programmer you will be. Knowing the concepts of different languages will help you in all programming, because you can match the programming language to the problem better, and you have a larger set of tools to work with. Even if certain features aren't directly supported in the language you are using, often they can be simulated. However, if you don't have a broad experience with languages, you won't know of all the possibilities you have to choose from.

Your First C Program

Here is your first C program, which prints "Hello world" to the screen and exits. Type it in, and give it the name Hello-World.c

```
#include <stdio.h>

/* PURPOSE:  This program is mean to show a basic */
/*           C program.  All it does is print     */
```

```
/*           "Hello World!" to the screen and     */
/*           exit.                                */

/* Main Program */
int main(int argc, char **argv)
{
 /* Print our string to standard output */
 puts("Hello World!\n");

 /* Exit with status 0 */
 return 0;
}
```

As you can see, it's a pretty simple program. To compile it, run the command

```
gcc -o HelloWorld Hello-World.c
```

To run the program, do

```
./HelloWorld
```

Let's look at how this program was put together.

Comments in C are started with `/*` and ended with `*/`. Comments can span multiple lines, but many people prefer to start and end comments on the same line so they don't get confused.

`#include <stdio.h>` is the first part of the program. This is a *preprocessor directive*. C compiling is split into two stages - the preprocessor and the main compiler. This directive tells the preprocessor to look for the file `stdio.h` and paste it into your program. The preprocessor is responsible for putting together the text of the program. This includes sticking different files together, running macros on your program text, etc. After the text is put together, the preprocessor is done and the main compiler goes to work.

Now, everything in `stdio.h` is now in your program just as if you typed it there yourself. The angle brackets around the filename tell the compiler to look in it's standard paths for the file (`/usr/include` and `/usr/local/include`, usually). If it was in quotes, like `#include "stdio.h"` it would look in the current directory for the file. Anyway, `stdio.h` contains the declarations for the standard input and output functions and variables. These declarations tell the compiler what functions are available for input and output. The next few lines are simply comments about the program.

Then there is the line `int main(int argc, char **argv)`. This is the start of a function. C Functions are declared with their name, arguments and return type. This declaration says that the function's name is `main`, it returns an `int` (integer - 4 bytes long on the x86 platform), and has two arguments - an `int` called `argc` and a `char **` called `argv`. You don't have to worry about where the arguments are positioned on the stack - the C compiler takes care of that for you. You also don't have to worry about loading values into and out of registers because the compiler takes care of that, too.

The `main` function is a special function in the C language - it is the start of all C programs (much like `_start` in our assembly-language programs). It always takes two parameters. The first parameter is the number of arguments given to this command, and the second parameter is a list of the arguments that were given.

The next line is a function call. In assembly language, you had to push the arguments of a function onto the stack, and then call the function. C takes care of this complexity for you. You simply have to call the function with the parameters in parenthesis. In this case, we call the function `puts`, with a single parameter. This parameter is the character string we want to print. We just have to type in the string in quotations, and the compiler takes care of defining storage and moving the pointers to that storage onto the stack before calling the function. As you can see, it's a lot less work.

Finally our function returns the number `0`. In assembly language, we stored our return value in `%eax`, but in C we just use the `return` command and it takes care

of that for us. The return value of the `main` function is what is used as the exit code for the program.

As you can see, using high-level languages makes life much easier. It also allows our programs to run on multiple platforms more easily. In assembly language, your program is tied to both the operating system and the hardware platform, while in compiled and interpreted languages the same code can usually run on multiple operating systems and hardware platforms. For example, this program can be built and executed on x86 hardware running Linux®, Windows®, UNIX®, or most other operating systems. In addition, it can also run on Macintosh hardware running a number of operating systems.

Additional information on the C programming language can be found in Appendix E.

Perl

Perl is an interpreted language, existing mostly on Linux and UNIX-based platforms. It actually runs on almost all platforms, but you find it most often on Linux and UNIX-based ones. Anyway, here is the Perl version of the program, which should be typed into a file named `Hello-World.pl`:

```
#!/usr/bin/perl

print("Hello world!\n");
```

Since Perl is interpreted, you don’t need to compile or link it. Just run in with the following command:

```
perl Hello-World.pl
```

As you can see, the Perl version is even shorter than the C version. With Perl you don’t have to declare any functions or program entry points. You can just start

typing commands and the interpreter will run them as it comes to them. In fact this program only has two lines of code, one of which is optional.

The first, optional line is used for UNIX machines to tell which interpreter to use to run the program. The `#!` tells the computer that this is an interpreted program, and the `/usr/bin/perl` tells the computer to use the program `/usr/bin/perl` to interpret the program. However, since we ran the program by typing in `perl Hello-World.pl`, we had already specified that we were using the perl interpreter.

The next line calls a Perl builtin function, print. This has one parameter, the string to print. The program doesn't have an explicit return statement - it knows to return simply because it runs off the end of the file. It also knows to return 0 because there were no errors while it ran. You can see that interpreted languages are often focused on letting you get working code as quickly as possible, without having to do a lot of extra legwork.

One thing about Perl that isn't so evident from this example is that Perl treats strings as a single value. In assembly language, we had to program according to the computer's memory architecture, which meant that strings had to be treated as a sequence of multiple values, with a pointer to the first letter. Perl pretends that strings can be stored directly as values, and thus hides the complication of manipulating them for you. In fact, one of Perl's main strengths is it's ability and speed at manipulating text.

Python

The Python version of the program looks almost exactly like the Perl one. However, Python is really a very different language than Perl, even if it doesn't seem so from this trivial example. Type the program into a file named `Hello-World.py`. The program follows:

```
#!/usr/bin/python
```

```
print "Hello World"
```

You should be able to tell what the different lines of the program do.

Review

Know the Concepts

- What is the difference between an intepretted language and a compiled language?
- What reasons might cause you to need to learn a new programming language?

Use the Concepts

- Learn the basic syntax of a new programming language. Re-code one of the programs in this book in that language.
- In the program you wrote in the question above, what specific things were automated in the programming language you chose?
- Modify your program so that it runs 10,000 times in a row, both in assembly language and in your new language. Then run the `time` command to see which is faster. Which does come out ahead? Why do you think that is?
- How does the programming language's input/output methods differ from that of the Linux system calls?

Going Further

- Having seen languages which have such brevity as Perl, why do you think this book started you with a language as verbose as assembly language?
- How do you think high level languages have affected the process of programming?
- Why do you think so many languages exist?
- Learn two new high level languages. How do they differ from each other? How are they similar? What approach to problem-solving does each take?